

DBMS

MODULE – 1

**Prepared by Mushtaq Ahmed D.M Assistant Professor,
Dept. of ISE**

A solid blue horizontal bar spanning the entire width of the slide at the bottom.

MODULE 1

- ❖ INTRODUCTION TO DATABASES
 - ❖ OVERVIEW OF DATABASE LANGUAGES AND ARCHITECTURES
 - ❖ CONCEPTUAL DATA MODELLING USING ENTITIES AND RELATIONSHIPS
-

Basic Definitions

Database:

- A collection of related data.

Data:

- Known facts that can be recorded and have an implicit meaning.

Mini-world:

- Some part of the real world about which data is stored in a database. For example, student grades and transcripts at a university.

Database Management System (DBMS):

A software package/ system to facilitate the creation and maintenance of a computerized database.

Database System:

- The DBMS software together with the data itself. Sometimes, the applications are also included.

DATABASE MANAGEMENT SYSTEMS

The DBMS is a general-purpose software system that facilitates the processes of defining, constructing, manipulating, and sharing databases among various users and applications

Defining a Database

- * Involves specifying the data types, structures, and constraints of the data to be stored in the database.
- * The database definition or descriptive information is also stored by the DBMS in the form of a database catalog or dictionary; it is called meta-data.

Constructing the database

- * is the process of storing the data on some storage medium that is controlled by the DBMS

Manipulating a database

- * Includes functions such as querying the database to retrieve specific data
- * Updating the database to reflect changes in the miniworld
- * reports from the data.
- * Sharing a database allows multiple users and programs to access the database

An application program accesses the database by sending queries or requests for data to the DBMS. A query typically causes some data to be retrieved.

Transaction may cause some data to be read and some data to be written into the database.

Other important functions provided by the DBMS include

- *Protecting the database and maintaining it over a long period of time

Protection includes system protection against hardware or software malfunction (or crashes) and security protection against unauthorized or malicious access.

- * DBMS must be able to maintain the database system by allowing the system to evolve as requirements change over time

Simplified Database System Environment

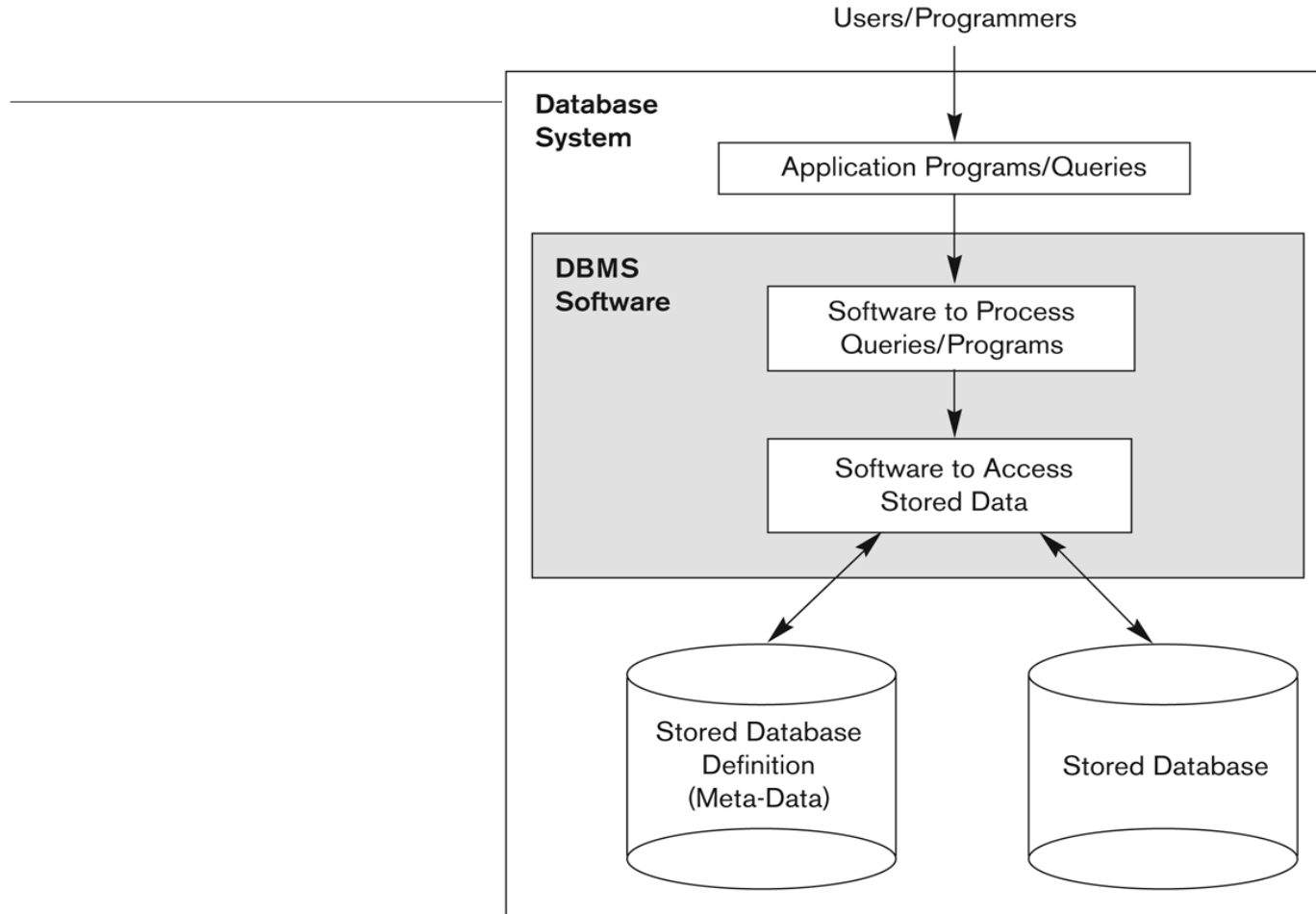


Figure 1.1
A simplified database system environment.

Typical DBMS Functionality

Define a particular database in terms of its data types, structures, and constraints

Construct or Load the initial database contents on a secondary storage medium

Manipulating the database:

- Retrieval: Querying, generating reports
- Modification: Insertions, deletions and updates to its content
- Accessing the database through Web applications

Processing and Sharing by a set of concurrent users and application programs – yet, keeping all data valid and consistent

Example of a Database (with a Conceptual Data Model)

Mini-world for the example:

- Part of a UNIVERSITY environment.

Some mini-world *entities*:

- STUDENTs
- COURSEs
- SECTIONs (of COURSEs)
- (academic) DEPARTMENTs
- INSTRUCTORs

Example of a Database (with a Conceptual Data Model)

Some mini-world *relationships*:

- SECTIONs *are of specific* COURSEs
- STUDENTs *take* SECTIONs
- COURSEs *have prerequisite* COURSEs
- INSTRUCTORs *teach* SECTIONs
- COURSEs *are offered by* DEPARTMENTs
- STUDENTs *major in* DEPARTMENTs

Note: The above entities and relationships are typically expressed in a conceptual data model, such as the ENTITY-RELATIONSHIP data mode

Example of a simple database

College Database

Student Entity Type

USN(KEY)	Name	Phone
Character(10)	Character(30)	integer

Student Entity Type

EID(KEY)	Name	Phone
Character(20)	Character(40)	integer

Staff Entity Type

STUDENT

Name	Student_number	Class	Major
Smith	17	1	CS
Brown	8	2	CS

COURSE

Course_name	Course_number	Credit_hours	Department
Intro to Computer Science	CS1310	4	CS
Data Structures	CS3320	4	CS
Discrete Mathematics	MATH2410	3	MATH
Database	CS3380	3	CS

SECTION

Section_identifier	Course_number	Semester	Year	Instructor
85	MATH2410	Fall	04	King
92	CS1310	Fall	04	Anderson
102	CS3320	Spring	05	Knuth
112	MATH2410	Fall	05	Chang
119	CS1310	Fall	05	Anderson
135	CS3380	Fall	05	Stone

GRADE_REPORT

Student_number	Section_identifier	Grade
17	112	B
17	119	C
8	85	A
8	92	A
8	102	B
8	135	A

PREREQUISITE

Course_number	Prerequisite_number
CS3380	CS3320
CS3380	MATH2410
CS3320	CS1310

Figure 1.2
A database that stores student and course information.

USN(KEY)	Name	Phone
1a	defg	34567
1b	abc	34434
1c	efg	53475

Staff Entity Type

EID(KEY)	Name	Phone
2a	abc	3575
2b	abc	3575
2c	def	4567

Example of a simplified database catalog

RELATIONS

Relation_name	No_of_columns
STUDENT	4
COURSE	4
SECTION	5
GRADE_REPORT	3
PREREQUISITE	2

Figure 1.3

An example of a database catalog for the database in Figure 1.2.

COLUMNS

Column_name	Data_type	Belongs_to_relation
Name	Character (30)	STUDENT
Student_number	Character (4)	STUDENT
Class	Integer (1)	STUDENT
Major	Major_type	STUDENT
Course_name	Character (10)	COURSE
Course_number	XXXXNNNN	COURSE
....		
....		
....		
Prerequisite_number	XXXXNNNN	PREREQUISITE

Note: Major_type is defined as an enumerated type with all known majors. XXXXNNNN is used to define a type with four alpha characters followed by four digits

[BACK](#)

Main Characteristics of the Database Approach

The main characteristics of the database approach versus the file-processing

Self-describing nature of a database system:

- A DBMS **catalog** stores the *description* of the database. The description is called **meta-data**. This allows the DBMS software to work with different databases.

Insulation between programs and data:

- **Program – data independence**
 - Allows changing data storage structures and operations without having to change the DBMS access programs.
- **Program – operation independence**
 - User application programs can operate on the data by invoking operations through their names and parameters regardless how operations are implemented.

Main Characteristics of the Database Approach

Data Abstraction:

- The characteristics that allows program – data independence and program – operation independence is called data abstraction.
- A **data model** is used to hide storage details and present the users with a *conceptual view* of the database.

Support of multiple views of the data:

- Each user may see a different view of the database, which describes *only* the data of interest to that user.

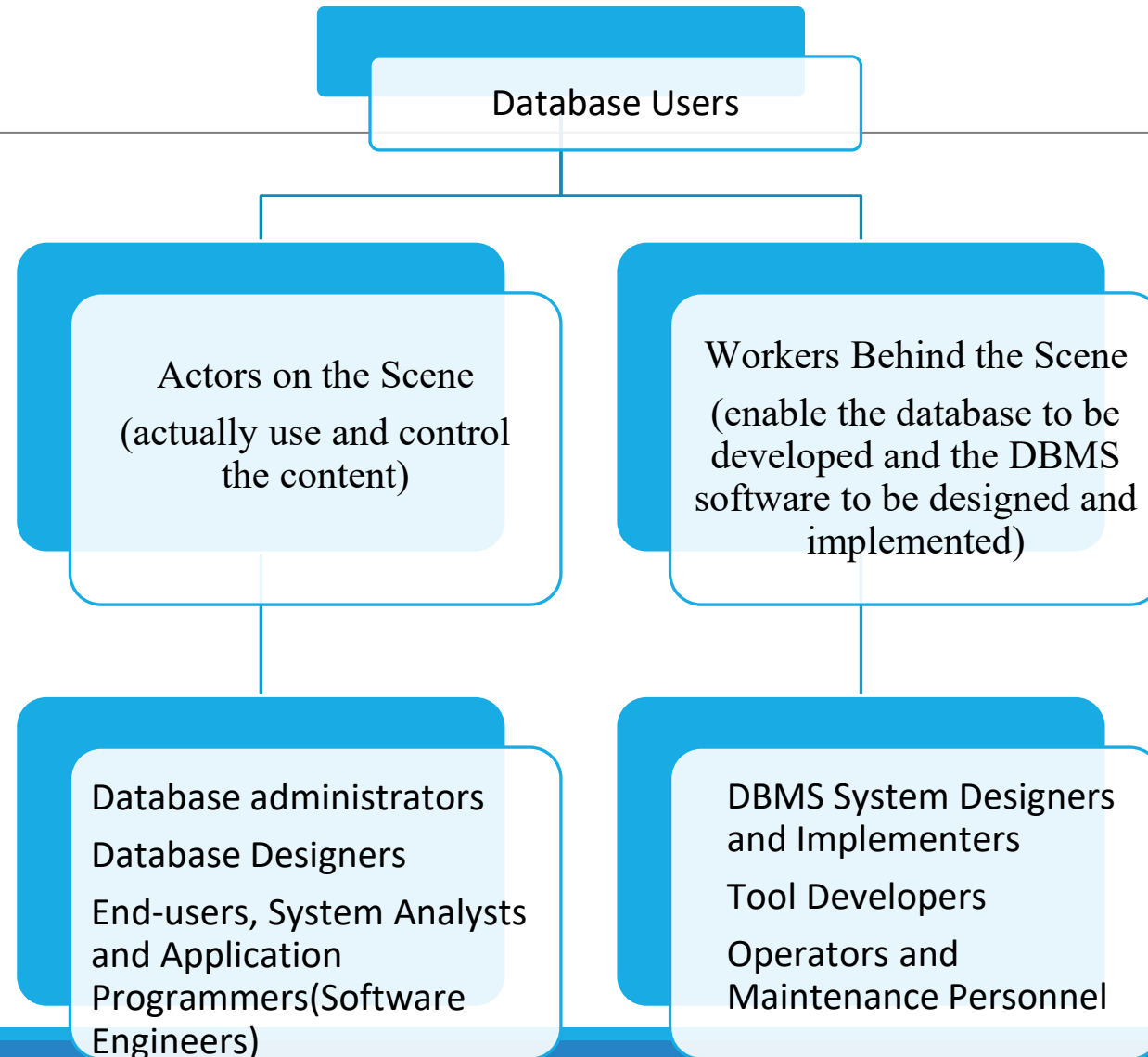
Sharing of data and multiuser transaction processing :

- Allowing a set of concurrent users to retrieve and to update the database. Concurrency control within the DBMS guarantees that each **transaction** is correctly executed or completely aborted.
- OLTP (Online Transaction Processing) is a major part of database applications

Differentiate between File System and DBMS

File System	DBMS
In traditional file processing, data definition is typically part of the application programs themselves.	A fundamental characteristic of the database approach is that the database system contains not only the database itself but also a complete definition or description of the database structure and constraints.
In traditional file processing, the structure of data files is embedded in the application programs, so any changes to the structure of a file may require changing all programs that access that file.	DBMS access programs do not require such changes in most cases. The structure of data files is stored in the DBMS catalog separately from the access programs.
Does not support multiple view of data	A database typically has many users, each of whom may require a different perspective or view of the database.
Concurrent access to the data in the file system has many problems like Reading the file while other deleting some information, updating some information etc.	DBMS takes care of Concurrent access using some form of locking.

Database Users



Database Users

Actors on the Scene

- **Database administrators**: responsible for authorizing access to the database, for co – ordinating and monitoring its use, acquiring software, and hardware resources, controlling its use and monitoring efficiency of operations.
- **Database Designers**: responsible to define the content, the structure, the constraints, and functions or transactions against the database. They must communicate with the end-users and understand their needs.
- **End-users**: they use the data for queries, reports and some of them actually update the database content.
- **System Analysts and Application Programmers(Software Engineers):**

System Analyst determine the requirements of end users, especially Naïve and parametric end users and develop specifications for canned transactions that meet these requirements.

Application Programmers implement these specifications as programs, then they test, debug, document and maintain these canned transactions.

Categories of End-users

Casual : access database occasionally when needed, use query languages

Naive or Parametric : They use previously well-defined functions in the form of “canned transactions” against the database. Examples are bank-tellers or reservation clerks who do this activity for an entire shift of operations.

Sophisticated : these include business analysts, scientists, engineers, others thoroughly familiar with the system capabilities. Many use tools in the form of software packages that work closely with the stored database.

Stand-alone : mostly maintain personal databases using ready-to-use packaged applications. An example is a tax program user that creates his or her own internal database.

Database Users

Workers behind the scene

- **DBMS System Designers and Implementers**
 - Design and implement the DBMS modules and interfaces as a software package
- **Tool Developers**
 - Design and implement tools—the software packages that facilitate database modeling and design, database system design, and improved performance ex: tools for performance monitoring
- **Operators and Maintenance Personnel**
 - (system administration personnel) are responsible for the actual running and maintenance of the hardware and software environment for the database system

Advantages of Using the Database Approach

➤ Controlling redundancy in data storage and in development and maintenance efforts.

- In file processing, every user group maintains its own files for handling its data-processing applications, many of the data being repeated(redundant)
- This redundancy(**controlled redundancy**) in storing the same data multiple times leads to several problems
 - (i) Duplication of effort
 - (ii) Storage space is wasted
 - (iii) Inconsistent Data

➤ Restricting unauthorized access to data

- Provide a security and authorization subsystem, which the DBA uses to create accounts and to specify account restrictions(**privileged software**)

Advantages of Using the Database Approach

➤ Providing persistent storage for program Objects

- The values of program variables or objects are discarded once a program terminates, unless the programmer explicitly stores them in permanent files
- In this it can be stored permanently in an object-oriented DBMS.
- Such an object is said to be persistent, since it survives the termination of program execution

➤ Providing Storage Structures for efficient Query Processing

- Provide capabilities for efficiently executing queries and updates
- Provide specialized data structures and search techniques to speed up query execution
- Query processing and optimization module of the DBMS is responsible for choosing an efficient query execution plan for each query based on the existing storage structures

Advantages of Using the Database Approach

➤ Providing backup and recovery services

- Backup and recovery subsystem provide facilities for recovering from hardware or software failures
- Computer system fails in the middle of a complex update transaction, database to be restored to the state

it was in before the transaction started executing

- Disk backup

➤ Providing multiple interfaces to different classes of users

- Provide a variety of user interfaces.
- Ex: Query languages for casual users, Programming language interfaces for application programmers

Advantages of Using the Database Approach

➤ Representing complex relationships among data

- Represent a variety of complex relationships among the data
-

- Ex: Brown

➤ Enforcing integrity constraints on the database

- Integrity constraints that must hold for the data
- Ex: (i) Specifying a data type for each data item, USN character(25)
(ii) Referential integrity, each USN to be associated with Fee details
(iii) Key or uniqueness , USN can not be repeated

➤ Permitting Inferencing and Actions using rules

- Deductive database systems Ex: Determining Year Back students based on university rules
- Active database/Triggers(a form of a rule activated by updates to the table) with tables Ex: Check if manager's salary greater than person working under him, if so, send a message

Data Models

- **Data Model:** A set of concepts to describe the *structure* of a database, and certain *constraints* that the database should obey.
- **Data Model Operations:** Operations for specifying database retrievals and updates by referring to the concepts of the data model. Operations on the data model may include *basic operations* and *user-defined operations*
- **Categories of data models**
 - i. Conceptual (high-level, semantic) data models**

Provide concepts that are close to the way many users *perceive* data. (Also called **entity-based** or **object-based** data models.)
 - ii. Physical (low-level, internal) data models**

Provide concepts that describe details of how data is stored in the computer
 - iii Implementation (representational) data models**

Provide concepts that fall between the above two, balancing user views with some computer storage details.

Schemas, Instances and Database State

- **Database Schema**

The *description* of a database. Includes descriptions of the database structure and the constraints that should hold on the database.

- **Schema Diagram**

A diagrammatic display of (some aspects of) a database schema.

- **Schema Construct**

A component of the schema or an object within the schema, e.g., STUDENT, COURSE.

- **Database State**

The actual data stored in a database at a *particular moment in time*. Also called **snapshot or instance or occurrence**.

(i) Initial Database State: Refers to the database when it is loaded

(ii) Valid State: A state that satisfies the structure and constraints of the database.

(iii) Distinction: The **database schema** changes *very infrequently*. The **database state** changes *every time the database is updated*.

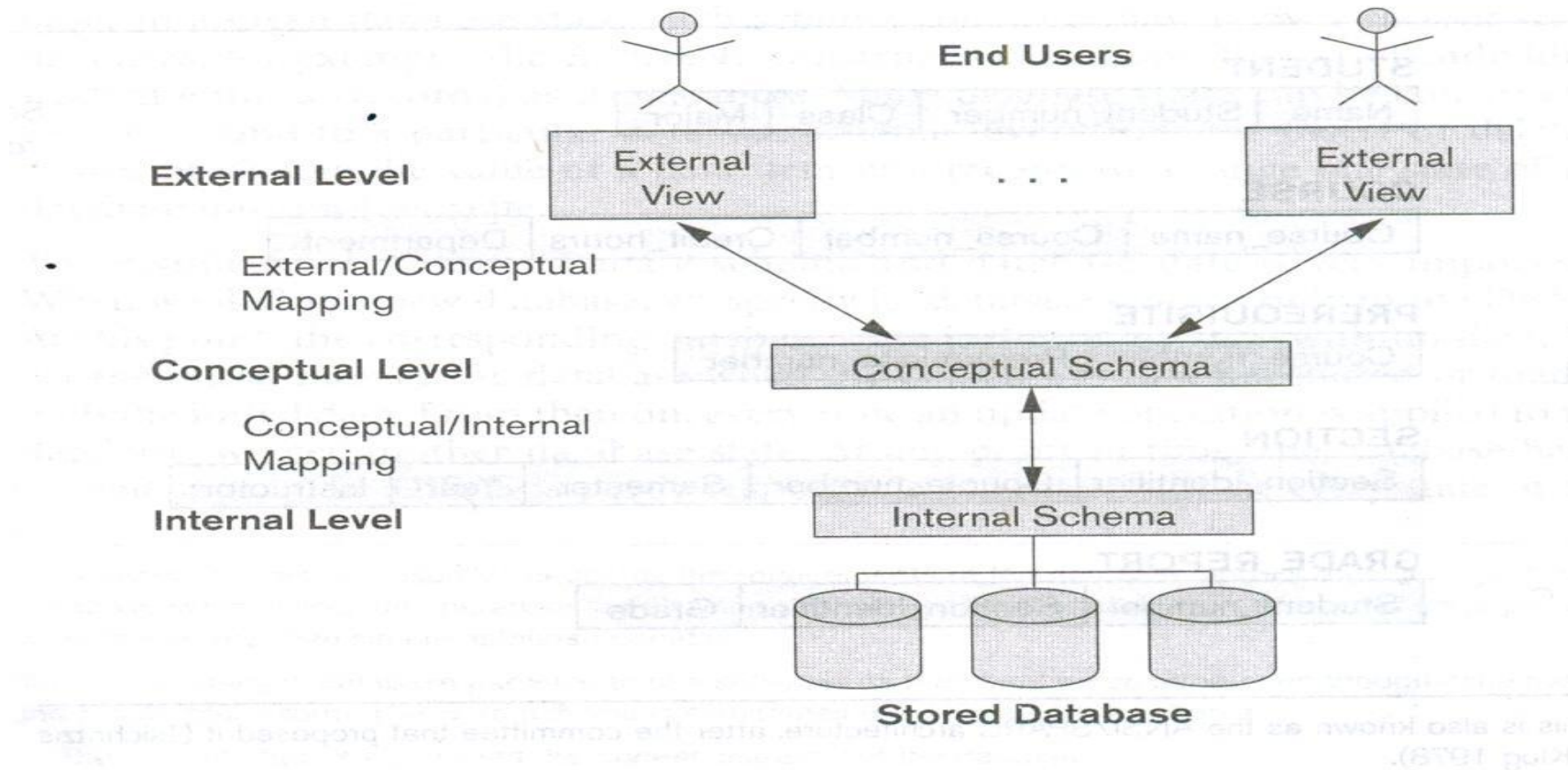
Schema is also called **intension**, whereas state is called **extension**.

Three-Schema Architecture

- Proposed to support DBMS characteristics of
 - (i) Program-data independence.
 - (ii) Support of multiple views of the data.

- Defines DBMS schemas at *three levels*:
 - **Internal schema** at the internal level to describe physical storage structures and access paths. Typically uses a *physical* data model.
 - **Conceptual schema** at the conceptual level to describe the structure and constraints for the *whole* database for a community of users. Uses a *conceptual* or an *implementation* data model.
 - **External schemas** at the external level to describe the various user views. Usually uses the same data model as the conceptual level.
 - **Mappings** among schema levels are needed to transform requests and data. Programs refer to an external schema, and are mapped by the DBMS to the internal schema for execution.

Three-Schema Architecture



Data Independence

The capacity to change the schema at one level of a database system without having to change the system at the next higher level.

Two types of data independences:

- **Logical Data Independence:** The capacity to change the conceptual schema without having to change the external schemas and their application programs.
- **Physical Data Independence:** The capacity to change the internal schema without having to change the conceptual schema.

When a schema at a lower level is changed, only the **mappings** between this schema and higher-level schemas need to be changed in a DBMS that fully supports data independence.

The higher-level schemas themselves are *unchanged*.

Hence, the application programs need not be changed since they refer to the external schemas.

Additional Implications of Using the Database Approach

Potential for enforcing standards:

- This is very crucial for the success of database applications in large organizations Standards refer to data item names, display formats, screens, report structures, meta-data (description of data) etc.

Reduced application development time:

- Incremental time to add each new application is reduced.

Flexibility to change data structures:

- Database structure may evolve as new requirements are defined.

Availability of up-to-date information:

- Very important for on-line transaction systems such as airline, hotel, car reservations.

Economies of scale:

- By consolidating data and applications across departments wasteful overlap of resources and personnel can be avoided.

Brief History of Database Applications

Early Database Applications: The Hierarchical and Network Models were introduced in mid 1960's and dominated during the seventies. A bulk of the worldwide database processing still occurs using these models.

Relational Model based Systems: The model that was originally introduced in 1970 was heavily researched and experimented with in IBM and the universities. Relational DBMS Products emerged in the 1980's.

Object-oriented applications: OODBMSs were introduced in late 1980's and early 1990's to cater to the need of complex data processing in CAD and other applications. Their use has not taken off much.

Data on the Web and E-commerce Applications: Web contains data in HTML (Hypertext markup language) with links among pages. This has given rise to a new set of applications and E-commerce is using new standards like XML (eXtended Markup Language).

Extending Database Capabilities

New functionality is being added to DBMSs in the following areas:

- Scientific Applications
- Image Storage and Management
- Audio and Video data management
- Data Mining
- Spatial data management
- Time Series and Historical Data Management

The above gives rise to new research and development in incorporating new data types, complex data structures, new operations and storage and indexing schemes in database systems.

When not to use a DBMS

Main inhibitors (costs) of using a DBMS:

- High initial investment and possible need for additional hardware.
- Overhead for providing generality, security, concurrency control, recovery, and integrity functions.

When a DBMS may be unnecessary:

- If the database and applications are simple, well defined, and not expected to change.
- If there are stringent real-time requirements that may not be met because of DBMS overhead.

If access to data by multiple users is not required.

When no DBMS may suffice:

- If the database system is not able to handle the complexity of data because of modeling limitations
- If the database users need special operations not supported by the DBMS.

DBMS Languages

- **Data Definition Language (DDL)**: Used by the DBA and database designers to specify the *conceptual schema* of a database. In many DBMSs, the DDL is also used to define internal and external schemas (views). In some DBMSs, separate **storage definition language (SDL)** and **view definition language (VDL)** are used to define internal and external schemas.
- **Data Manipulation Language (DML)**: Used to specify database retrievals and updates.
 - DML commands (**data sublanguage**) can be *embedded* in a general-purpose programming language (**host language**), such as COBOL, C or an Assembly Language.
 - Alternatively, *stand-alone* DML commands can be applied directly (**query language**).

Types of DML:

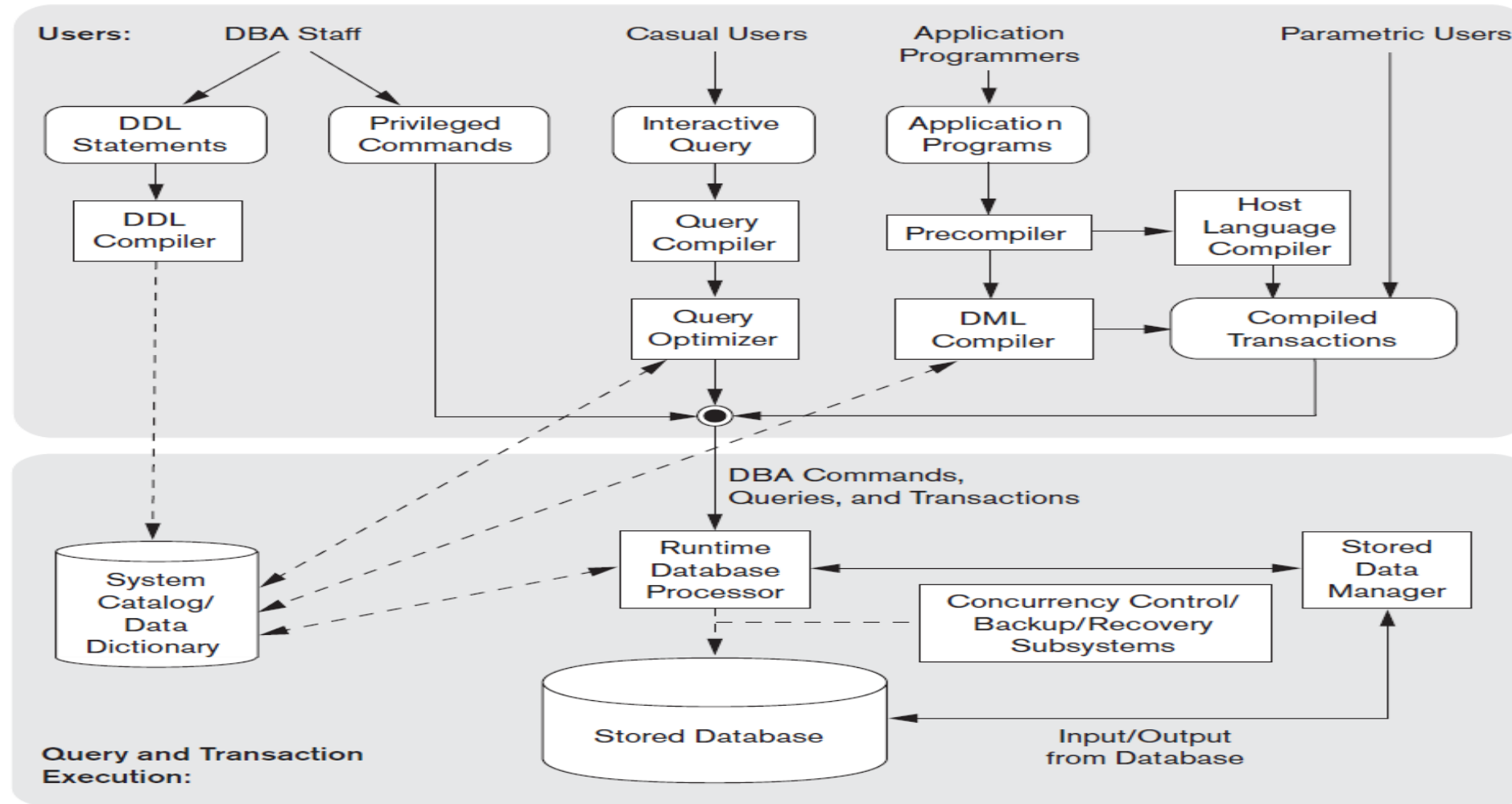
High Level or Non-procedural DML: e.g., SQL, are *set-oriented* and specify what data to retrieve than how to retrieve. Also called *declarative* languages.

Low Level or Procedural DML: record-at-a-time; they specify *how* to retrieve data and include constructs such as looping.

DBMS Interfaces

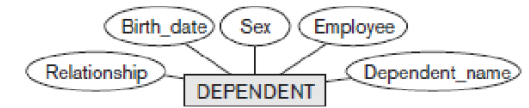
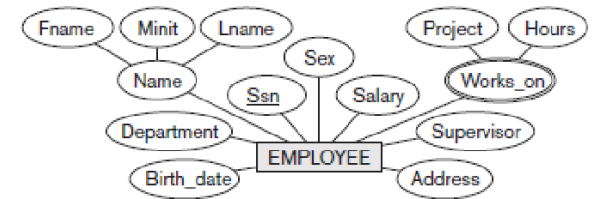
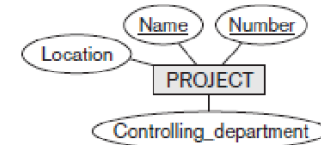
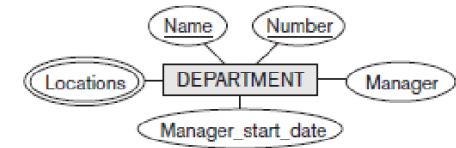
- * Stand-alone query language interfaces.
- * Programmer interfaces for embedding DML in programming languages
- * User-friendly interfaces:
 - Menu-based, popular for browsing on the web
 - Forms-based, designed for naïve users
 - Graphics-based (Point and Click, Drag and Drop etc.)
 - Natural language: requests in written English
 - Speech as Input and Output
 - Web Browser as an interface
 - Parametric interfaces (e.g., bank tellers) using function keys.
 - Interfaces for the DBA:
 - Creating accounts, granting authorizations
 - Setting system parameters
 - Changing schemas or access path

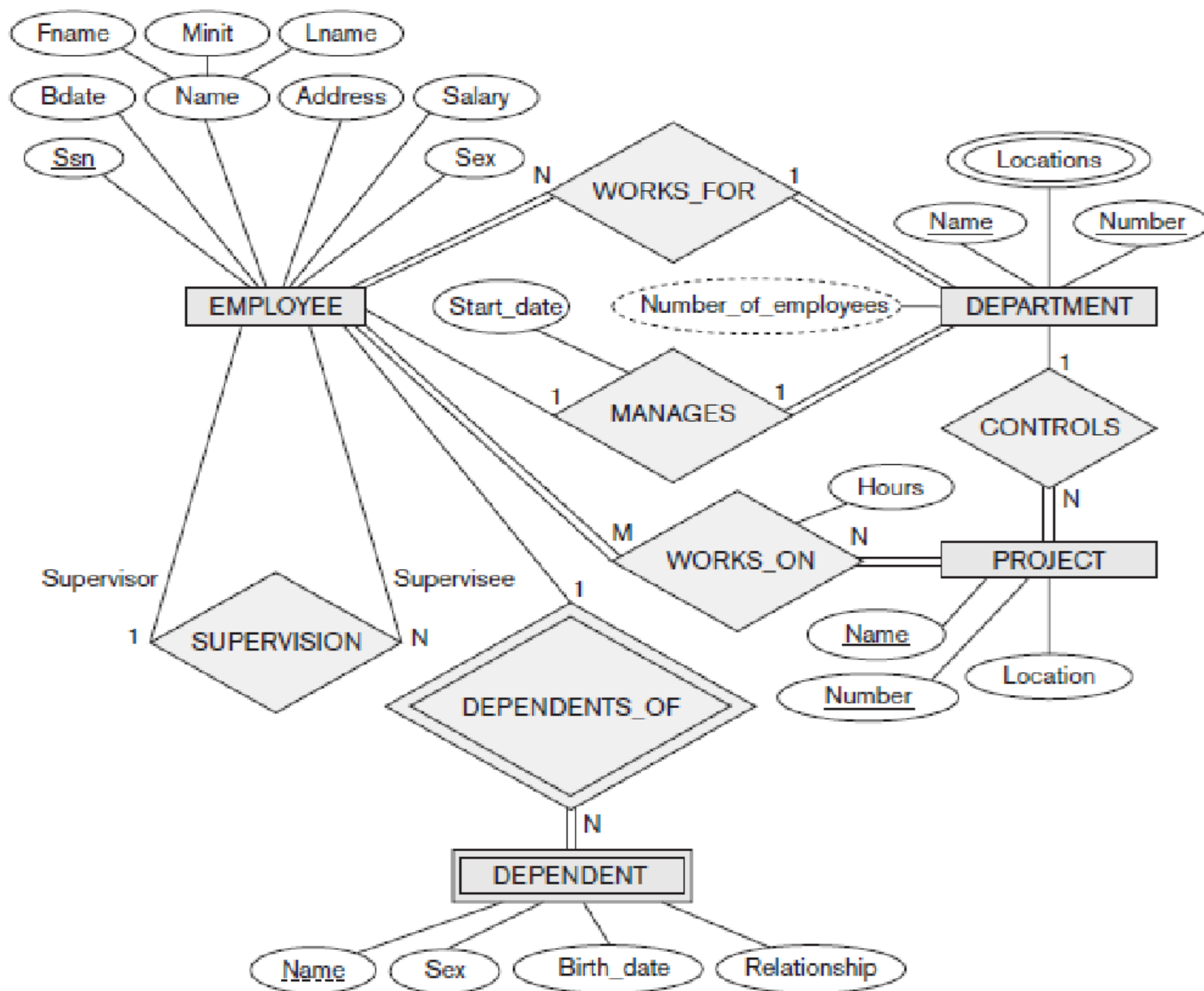
DBMS Component Modules



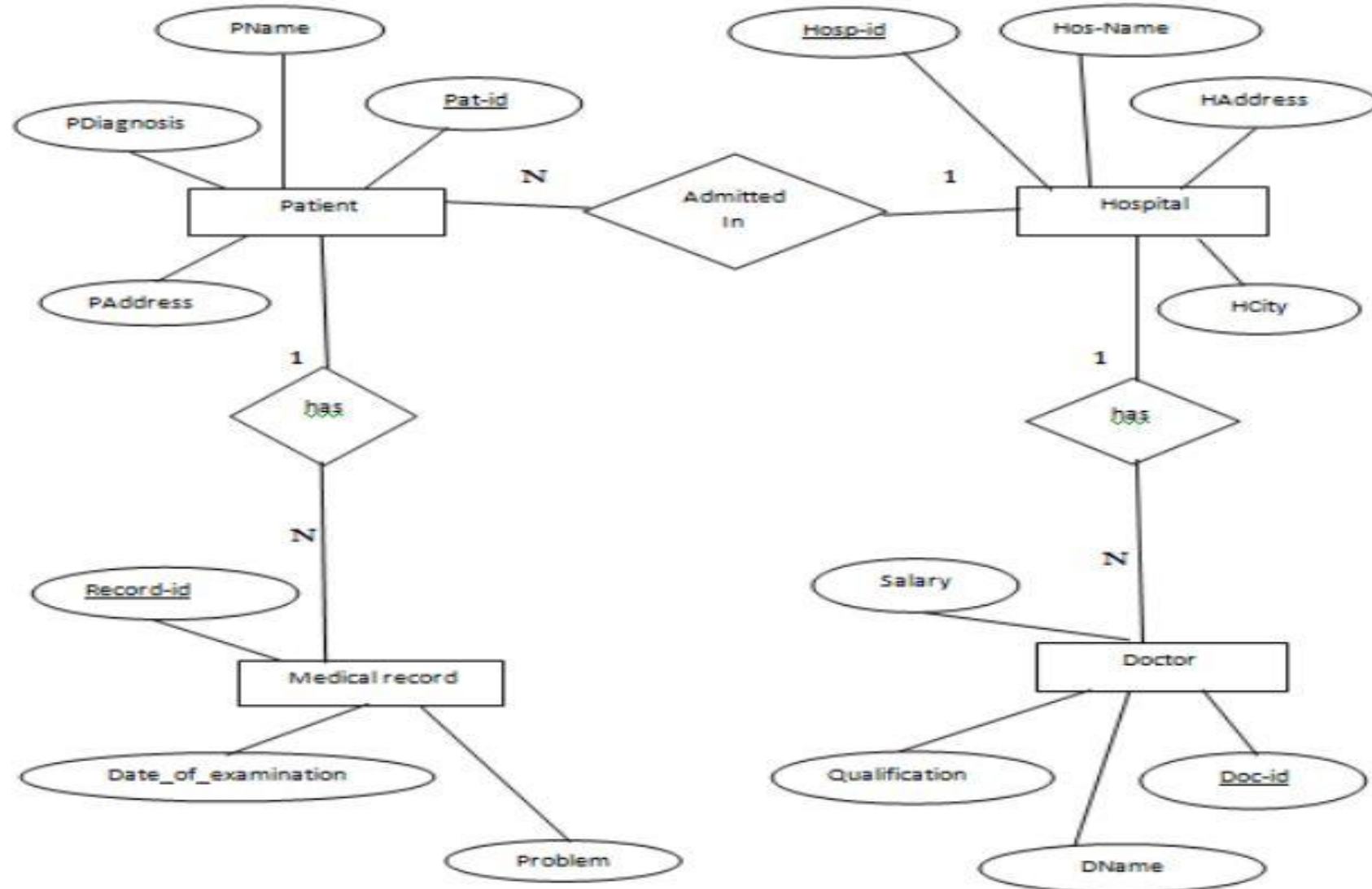
An Example Database Application

1. An entity type DEPARTMENT with attributes Name, Number, Locations, Manager, and Manager_start_date. Locations is the only multivalued attribute. We can specify that both Name and Number are (separate) key attributes because each was specified to be unique.
2. An entity type PROJECT with attributes Name, Number, Location, and Controlling_department. Both Name and Number are (separate) key attributes.
3. An entity type EMPLOYEE with attributes Name, Ssn, Sex, Address, Salary, Birth_date, Department, and Supervisor. Both Name and Address may be composite attributes; however, this was not specified in the requirements. We must go back to the users to see if any of them will refer to the individual components of Name—First_name, Middle_initial, Last_name—or of Address. In





Hospital Management System



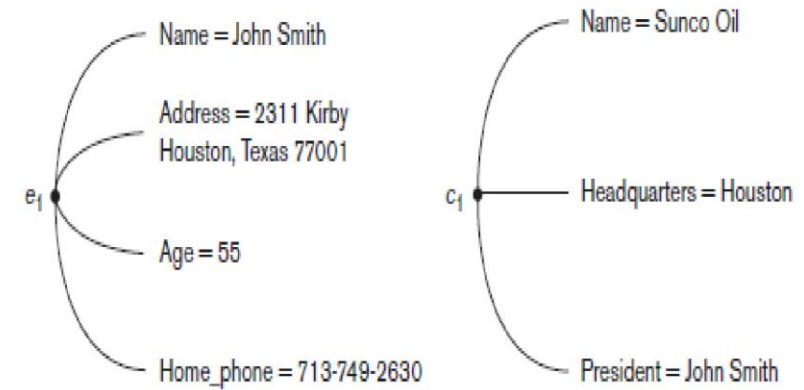
Entity Types, Entity Sets, Attributes and Keys

- **Entity**

- A thing in the real world with an independent existence.
- Eg: A particular person, Car, House or an Employee

- **Attribute**

- The particular property that describes the entity.
- Eg: An Employee's name, age, address, salary and job



Types of Attributes

(i) Composite versus simple (Atomic) Attributes

(ii) Single Valued versus Multivalued Attributes

(iii) Stored versus Derived Attributes

(iv) Complex Attributes

- For example, {Address_phone({Phone(Area_Code, Phone_Number))},
- Address(Street_Address (Number, Street, Apartment_Number), City, Zip_Code))

(v) Null Values

Entity Types

A collection of entities that have the same attributes.

Each entity type in the database is described by its name and attributes.

An entity type describes the **schema** or **intension** for a set of entities that share the same structure.

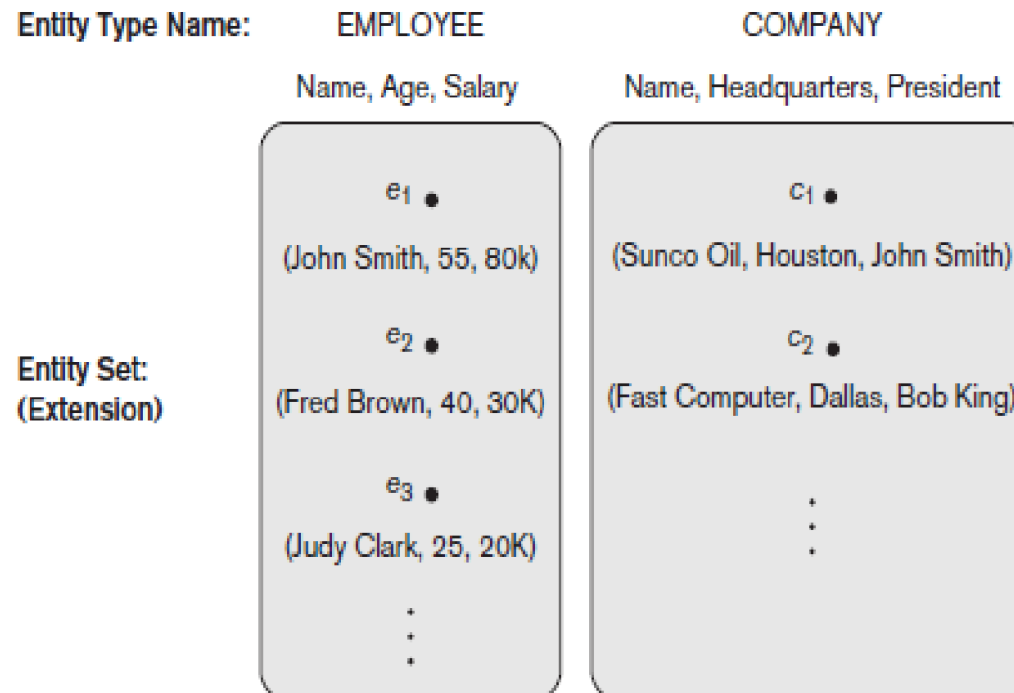
Eg: EMPLOYEE and COMPANY and a list of attributes for each.

An entity type is represented in ER diagrams as a rectangular box enclosing the entity type name.

Entity Set

The collection of entities of a particular entity type is grouped into an entity set.

This is also called as **Extension** of the entity type.



ENTITY SET corresponding to the ENTITY TYPE CAR

Entity Type

CAR

Registration(RegistrationNumber, State), VehicleID, Make, Model, Year, (Color)

Entity Set

car_1
((ABC 123, TEXAS), TK629, Ford Mustang, convertible, 1999, (red, black))
 car_2
((ABC 123, NEW YORK), WP9872, Nissan 300ZX, 2-door, 2002, (blue))
 car_3
((VSY 720, TEXAS), TD729, Buick LeSabre, 4-door, 2003, (white, blue))
.
.
.

Key Attribute of an Entity Type

An entity type usually has an attribute whose values are distinct for each individual entity in the entity set.

Such an attribute is called as key attribute and its values can be used to identify each entity uniquely.

Eg: No two companies will have the same name, so name attribute is the key attribute in COMPANY.

In ER diagrammatic notation, each key attribute has its name **underlined** inside the oval.

Value Sets (Domains) of Attributes

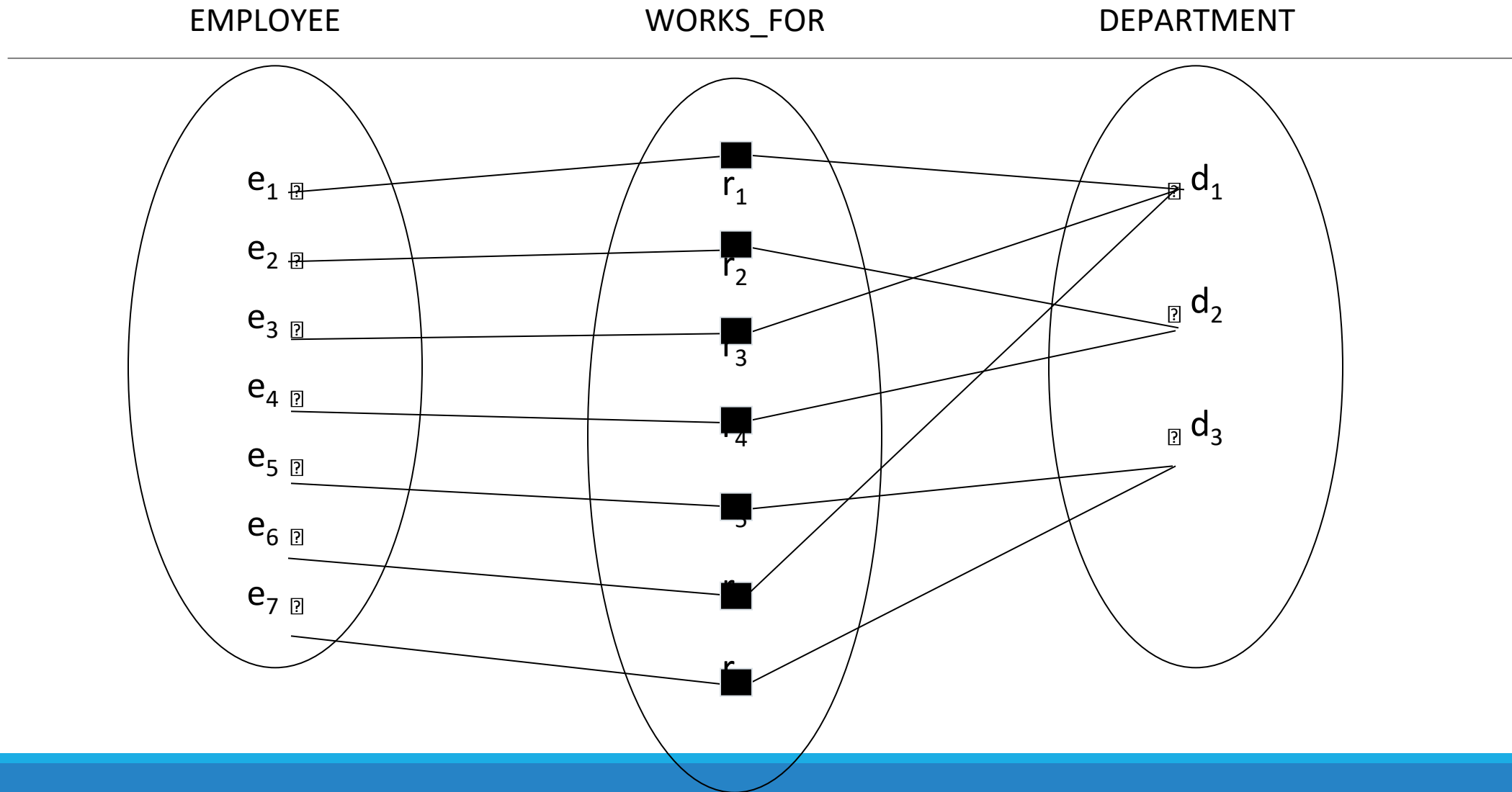
- Each simple attribute of an entity type is associated with a **value set** (or **domain** of values), which specifies the set of values that may be assigned to that attribute for each individual entity.
- Value sets are typically specified using the basic data types.
- Ex: If the range of ages allowed for employees is between 16 and 70, we can specify the value set of the Age attribute of EMPLOYEE to be the set of integer numbers between 16 and 70.
- We can specify the value set for the Name attribute to be the set of strings of alphabetic characters separated by blank characters, and so on.

Relationship Types, Relationship Sets, and Instances

A **relationship type** R among n entity types $E_1, E_2, \dots, E_n$ defines a set of associations – or a **relationship set** – among entities from these entity types.

Mathematically, the relationship set R is a set of **relationship instances** r_i , where each r_i associates n individual entities $(e_1, e_2, \dots, e_n)$ and each entity e_j in r_i is a member of entity set E_j , $1 \leq j \leq n$.

Example 1...

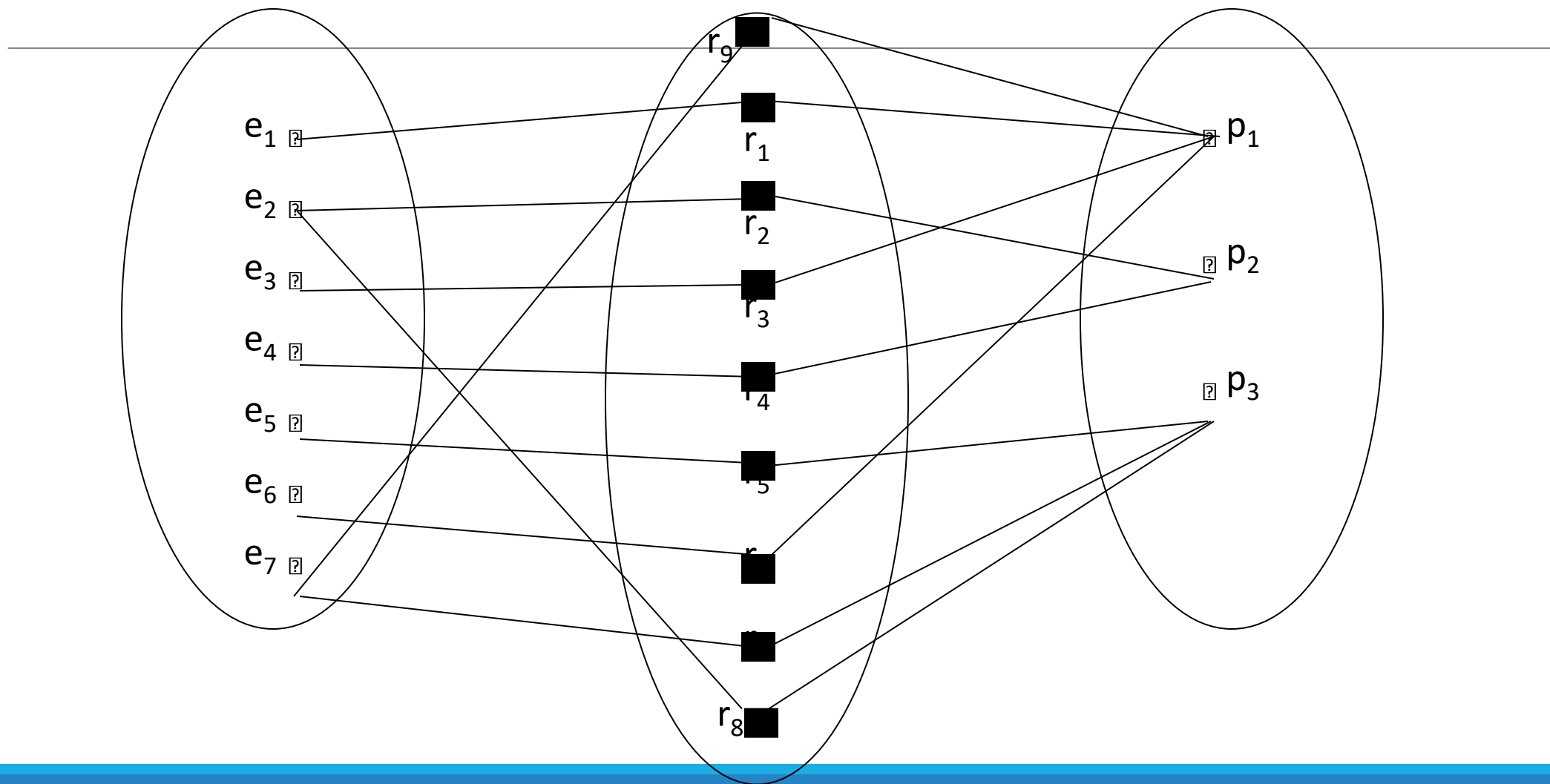


Example 2...

EMPLOYEE

WORKS_ON

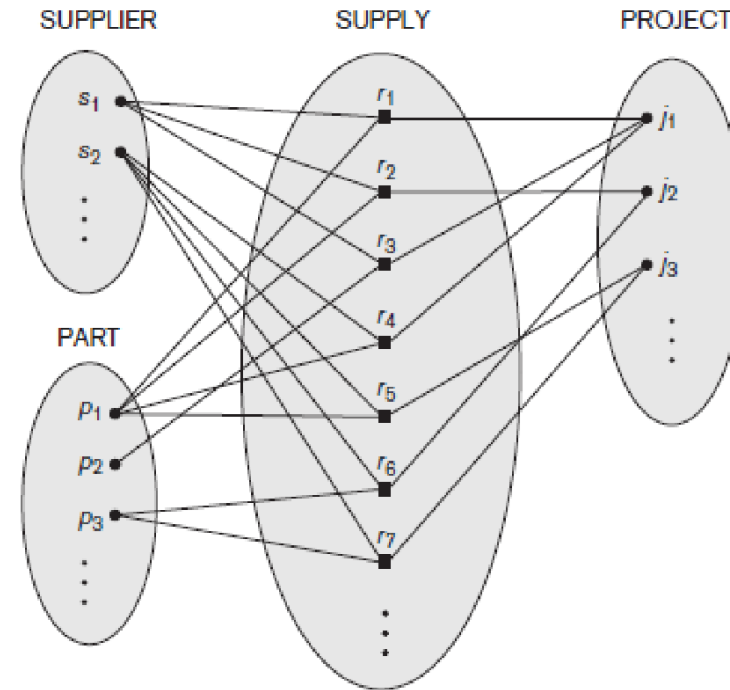
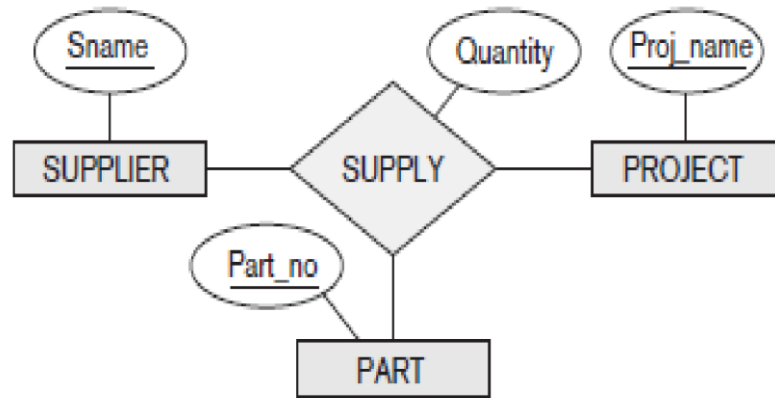
PROJECT



Relationship Degree

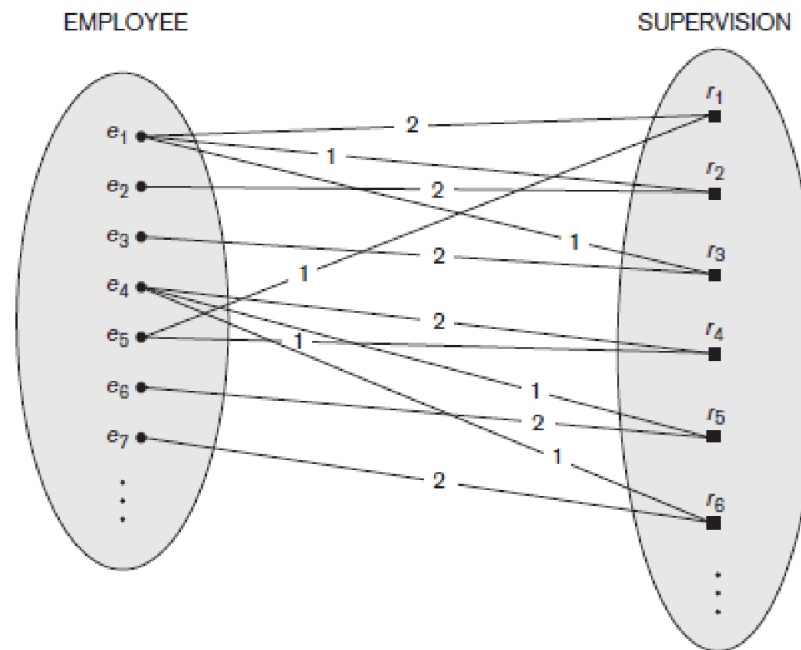
The degree of a relationship type is the number of participating entity types.

A relationship type of degree two is called **binary**, and one of degree three is called **ternary**.



Role Names and Recursive Relationships

- Each entity type that participates in a relationship type plays a particular **role** in the relationship.
- The role name signifies the role that a participating entity from the entity type plays in each relationship instance, and helps to explain what the relationship means.
- Some entity types participate more than once in a relationship type in different roles. Such relationship types are called **recursive relationships**.

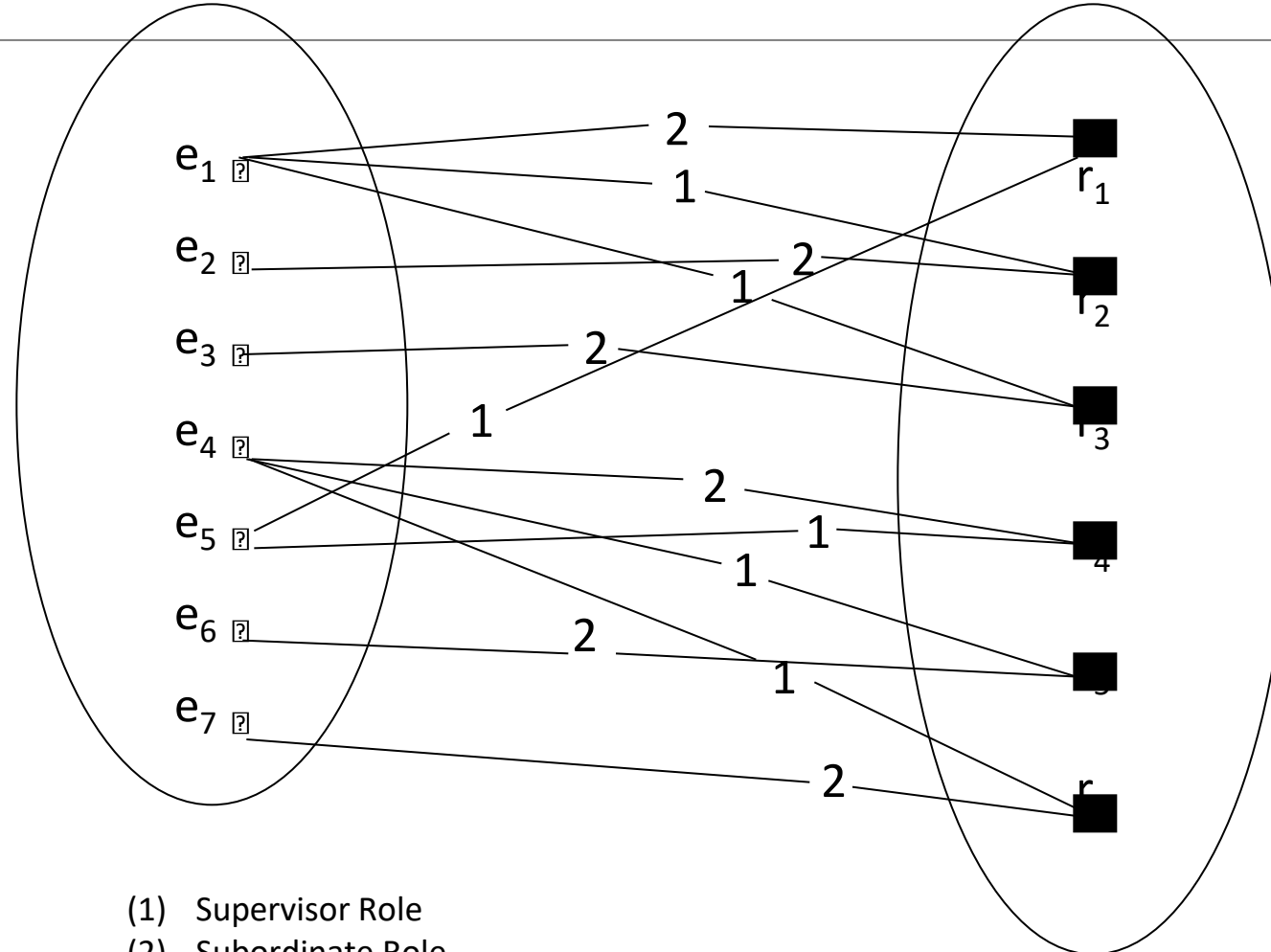


1-Supervisor
2-Supervisee

Example...

EMPLOYEE

SUPERVISION



- (1) Supervisor Role
- (2) Subordinate Role

Constraints on Binary Relationship Types

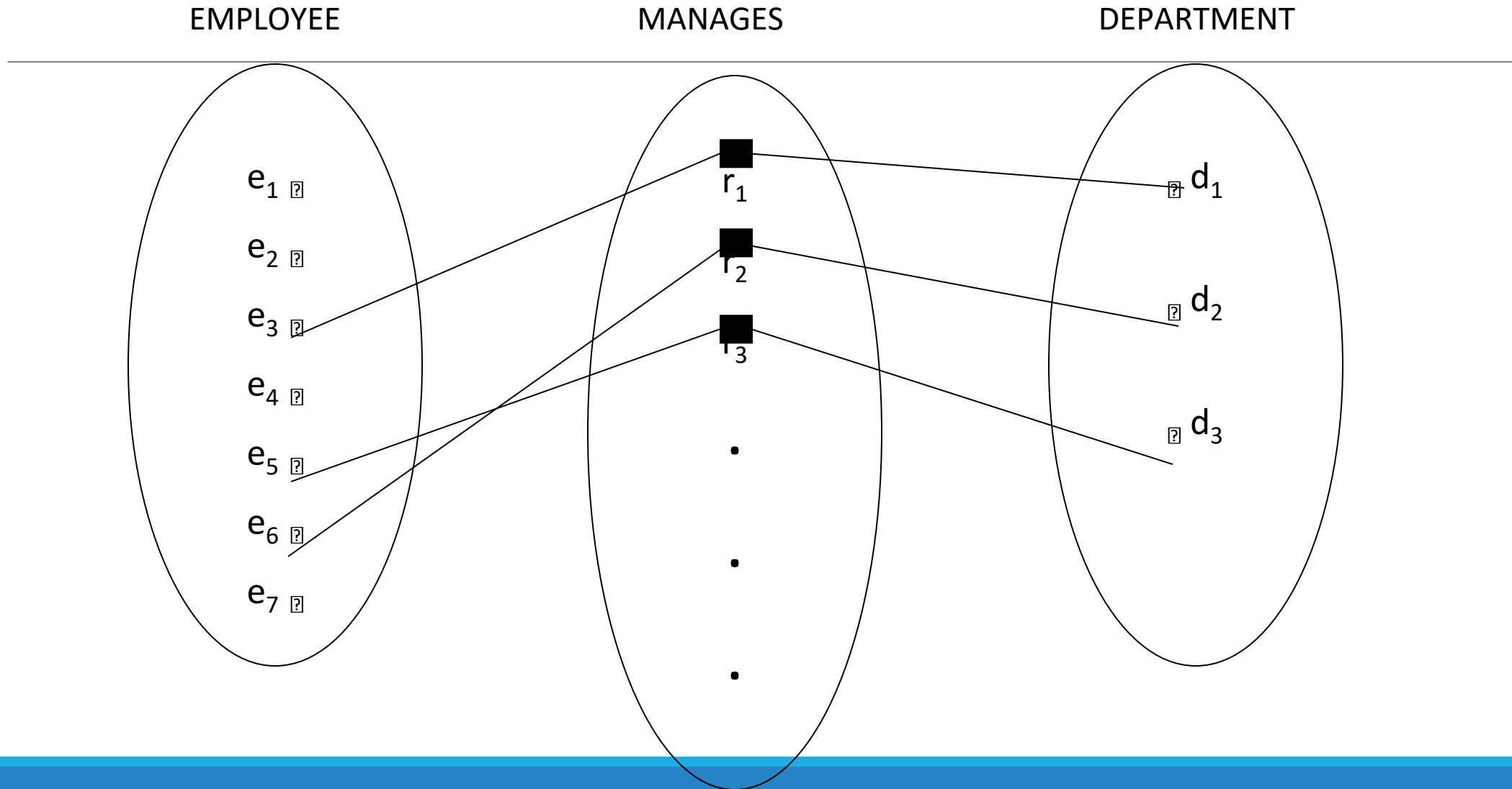
There are two types of relationship constraints:

- Cardinality ratio:

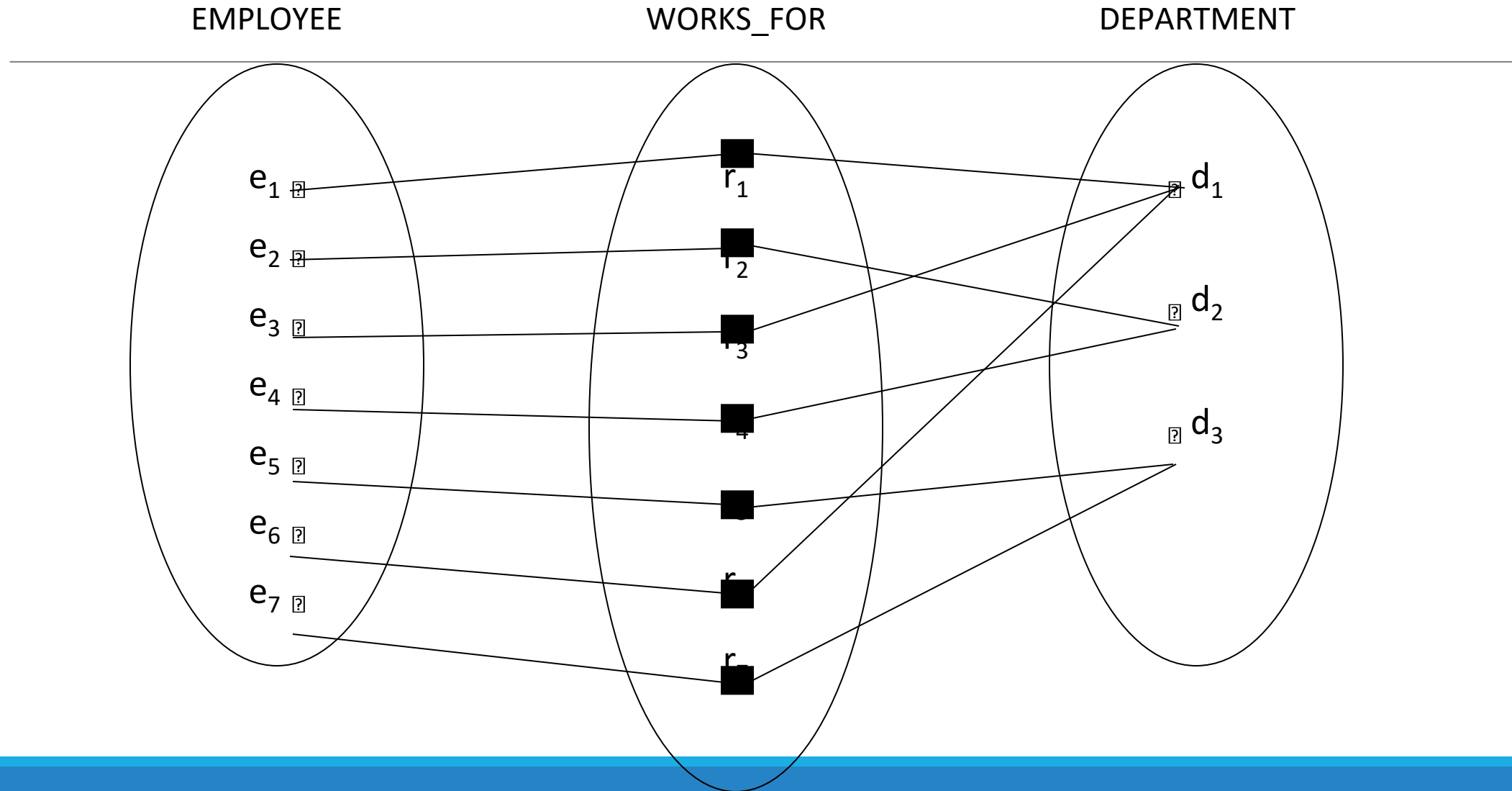
 - * The cardinality ratio for binary relationship specifies the maximum number of relationship instances that an entity can participate in.
 - * The possible cardinality ratio for binary relationship types are 1:1, 1:N, N:1 and M:N
- Participation:
 - * The participation constraint specifies whether the existence of an entity depends on its being related to another entity via the relationship type.
 - * This constraint specifies the minimum number of relationship instances that each entity can participate in.

The cardinality ratio and participation constraint together called as **structural constraint**.

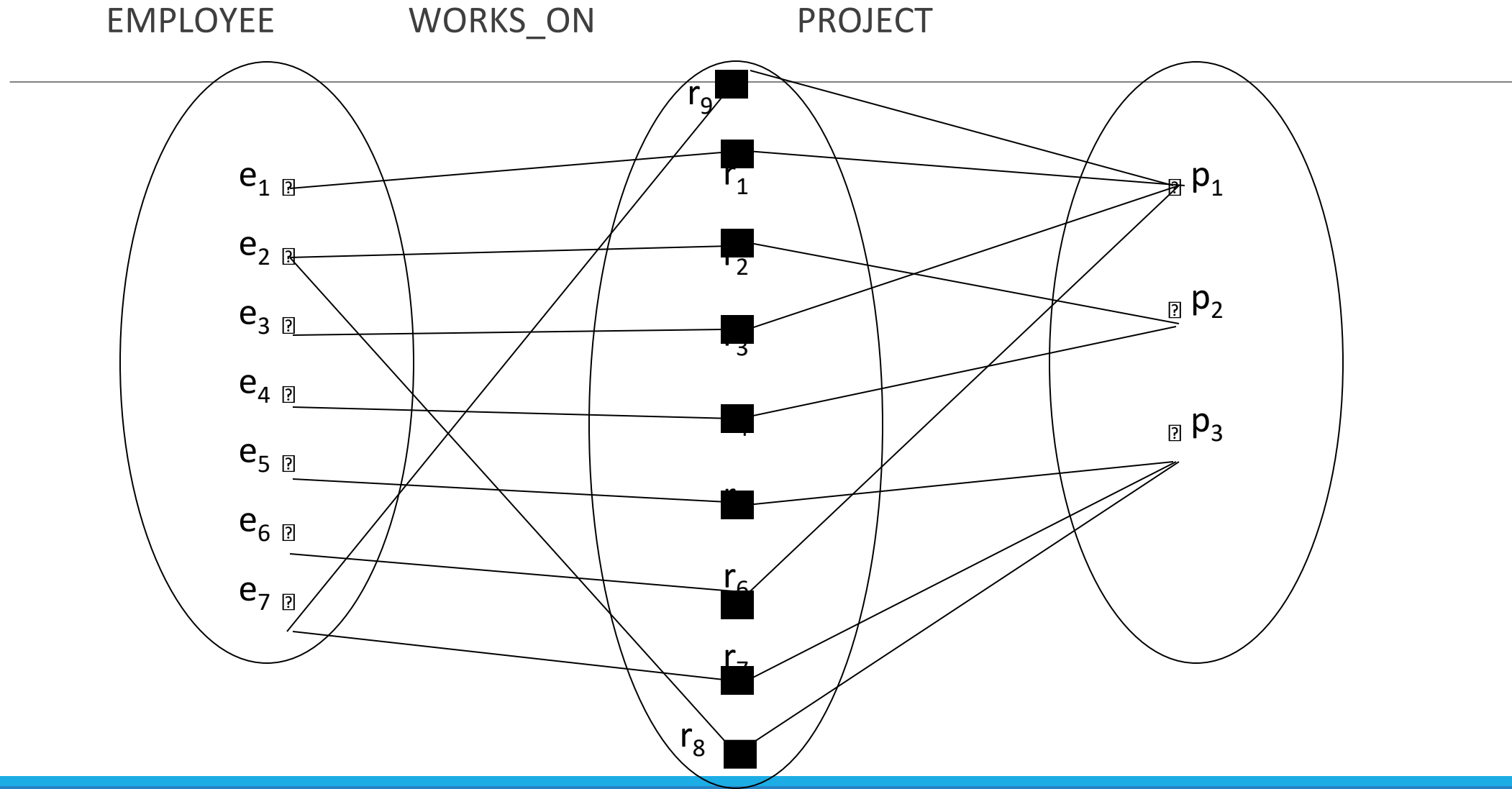
1:1 Relationship



1:N Relationship



M:N Relationship



Participation Constraint and Existence Dependence

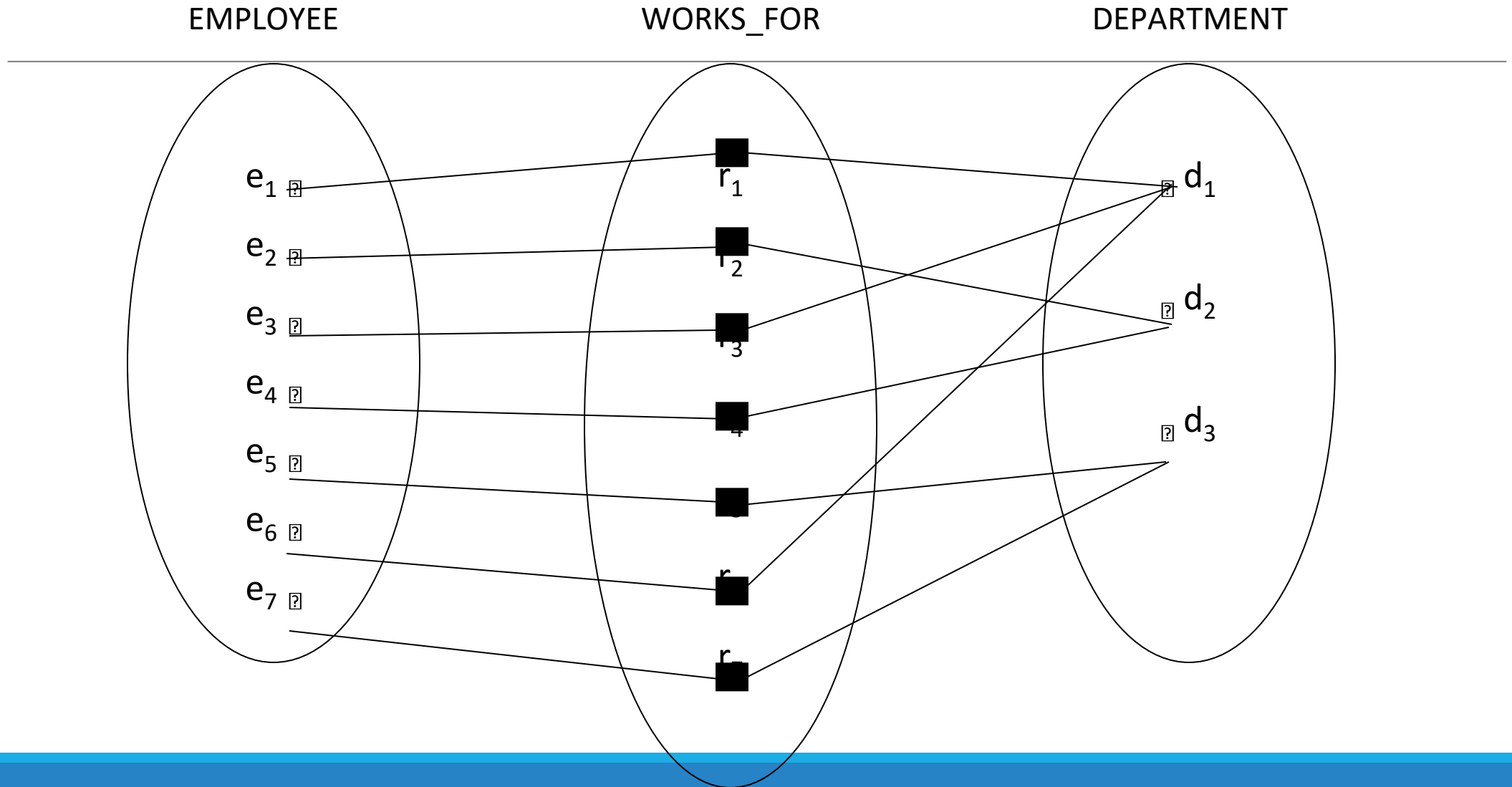
The participation constraint specifies whether the existence of an entity depends on its being related to another entity via the relationship type.

This constraint specifies the minimum number of relationship instances that each entity can participate in and is sometimes called the minimum cardinality constraint.

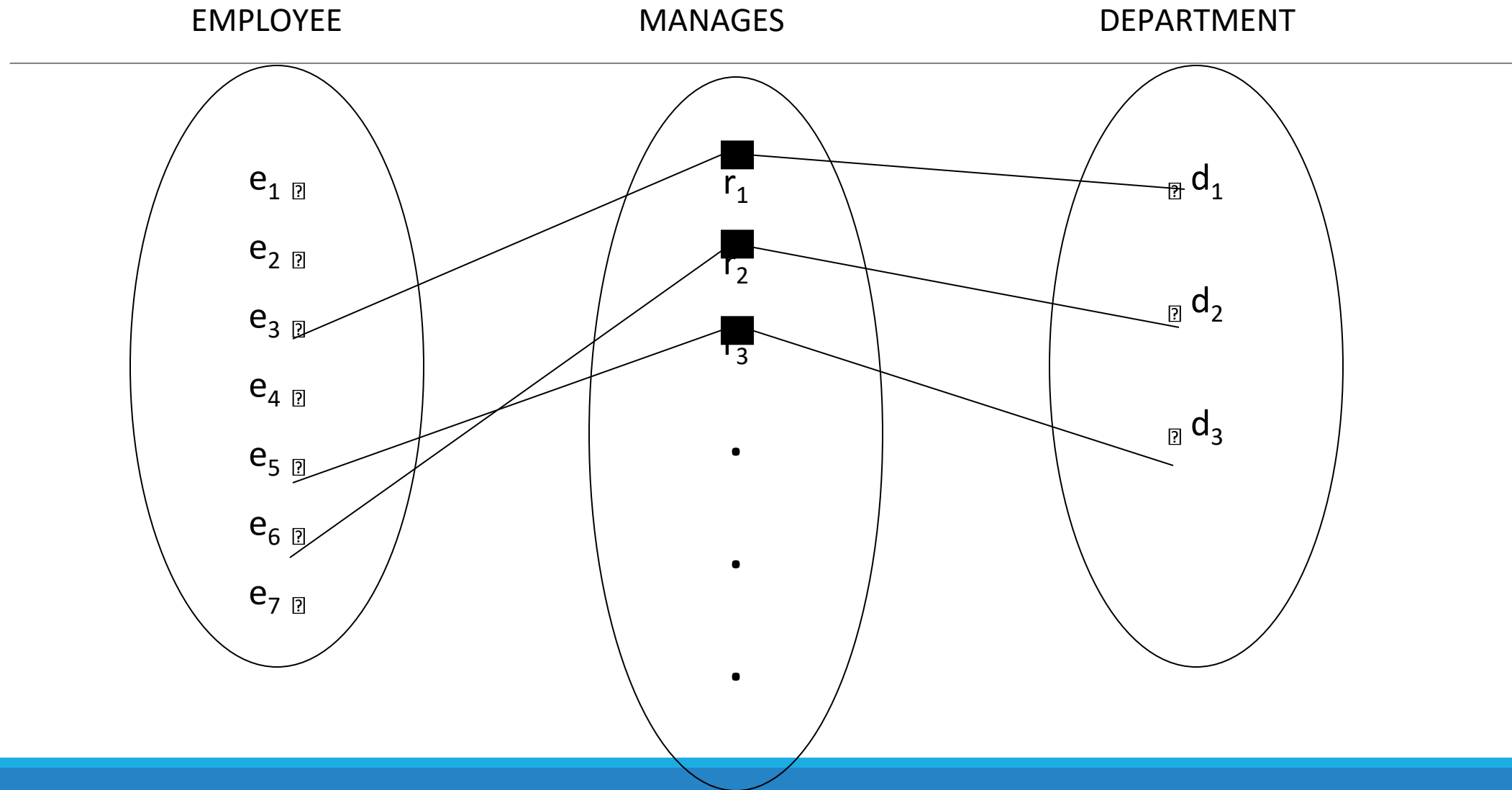
Two types of participation constraint:

- Total Participation, also called as Existence Dependency-If a company policy states that every employee must work for a department, then an employee entity can exist only if it participates in at least one WORKS_FOR relationship instance
- Partial Participation-we do not expect every employee to manage a department, so the participation of EMPLOYEE in the MANAGES relationship type is partial, meaning that some or part of the set of employee entities are related to some department entity via MANAGES, but not necessarily all.

Total Participation, also called as Existence Dependency



1:1 Relationship



Attributes as Relationship Types

A relationship type can have attributes; for example, HoursPerWeek of WORKS_ON; its value for each relationship instance describes the number of hours per week that an EMPLOYEE works on a PROJECT.

Weak Entity Types

Entity types that do not have key attributes of their own are called **weak entity types**.

The entity types which contain key attributes of their own are called **regular entity types** or **strong entity types**.

Entities belonging to weak entity type are identified by being related to specific entities from another entity type in combination with one of their attribute values. This other entity type is called as **identifying entity type** or **owner entity type**.

The relationship type that relates a weak entity type to its owner is called as **identifying relationship** of the weak entity type.

The weak entity type normally has a partial key, which is the set of attributes that can uniquely identify weak entities that are related to the same owner entity.

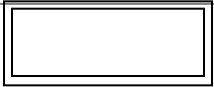
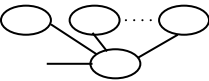
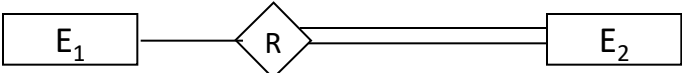
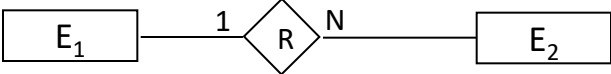
Example...

Example:

Suppose that a DEPENDENT entity is identified by the dependent's first name and birth date, *and* the specific EMPLOYEE that the dependent is related to.

DEPENDENT is a weak entity type with EMPLOYEE as its identifying entity type via the identifying relationship type DEPENDENT_OF.

Notations Used in ER Diagrams

Symbol	Meaning
	ENTITY TYPE
	WEAK ENTITY TYPE
	RELATIONSHIP TYPE
	IDENTIFYING RELATIONSHIP TYPE
	ATTRIBUTE
	KEY ATTRIBUTE
	MULTIVALUED ATTRIBUTE
	COMPOSITE ATTRIBUTE
	DERIVED ATTRIBUTE
	TOTAL PARTICIPATION OF E_2 IN R
	CARDINALITY RATIO 1:N FOR $E_1:E_2$ IN R
	STRUCTURAL CONSTRAINT (min, max) ON PARTICIPATION OF E IN R

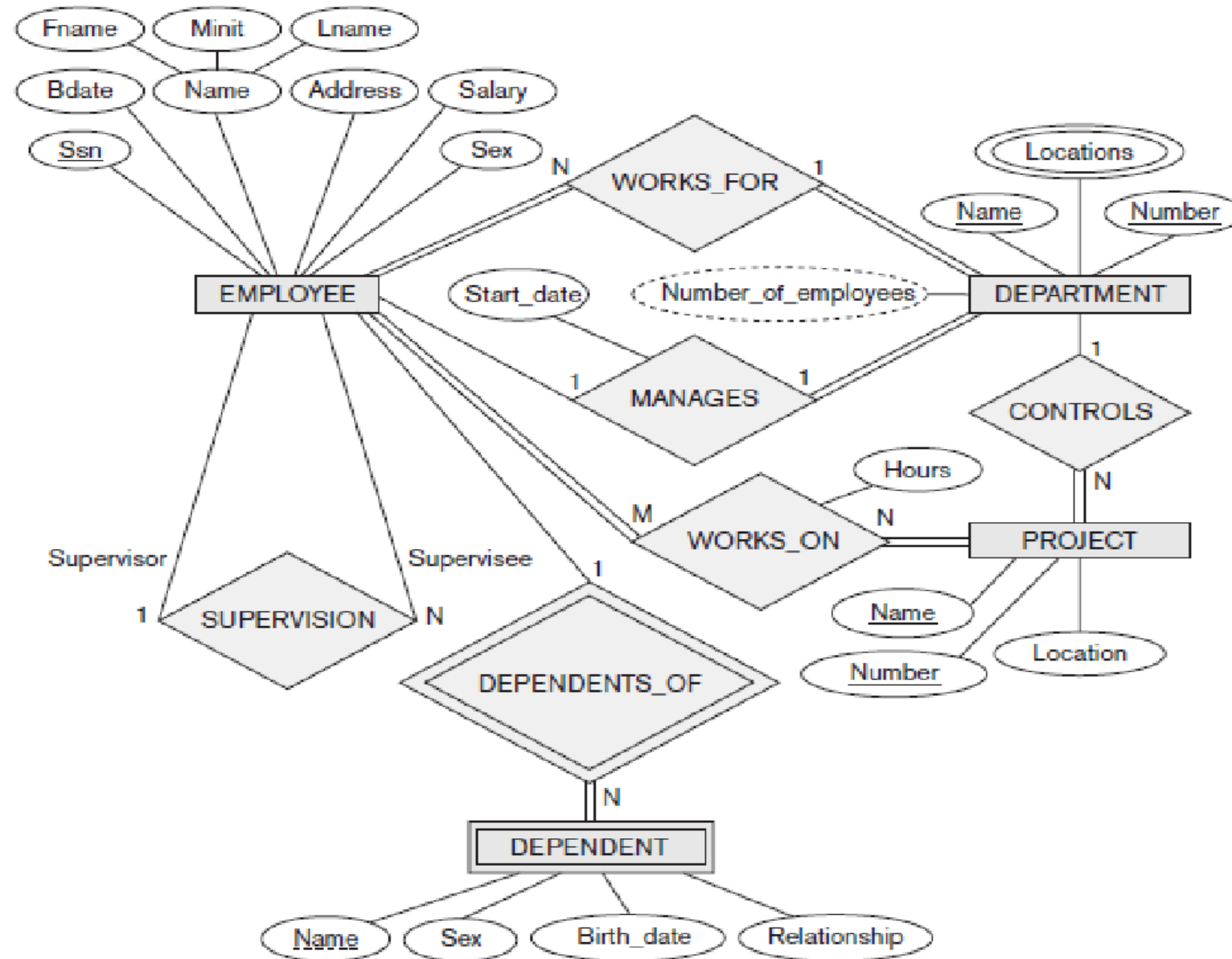
Proper Naming of Schema Constructs

Use singular names for entity types, rather than plural one.

Verbs tend to indicate the relationship types.

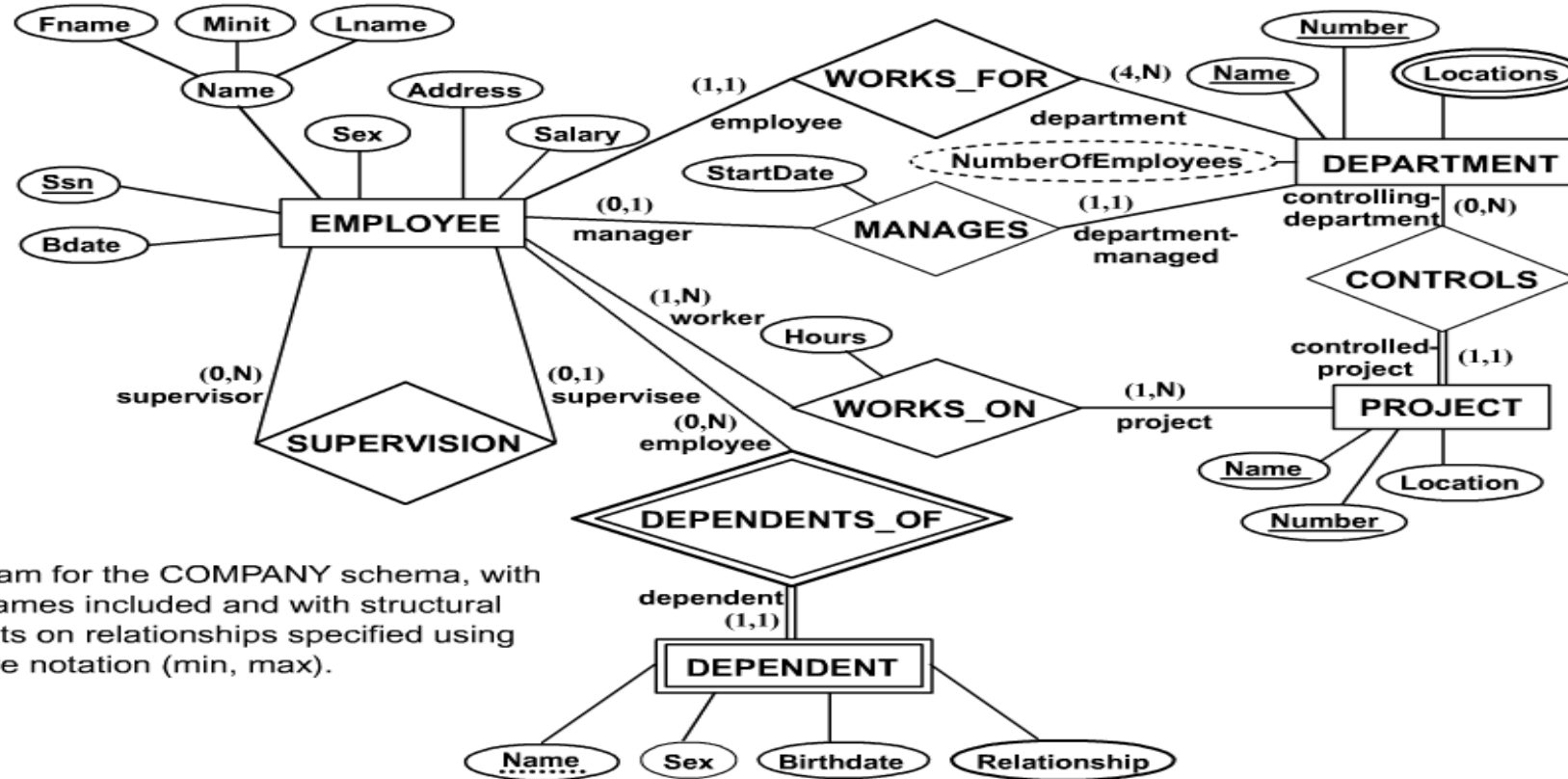
Another naming consideration involves choosing the binary relationship names to make the ER diagram of the schema readable from left to right and top to bottom.

E – R Diagram for COMPANY



Alternative Notations for ER Diagrams

Alternative ER Notations



ER diagram for the COMPANY schema, with all role names included and with structural constraints on relationships specified using alternative notation (min, max).